

Lecture 11 - Discriminant Functions, Mixture Models & Expectation-Maximization Algorithm

Solving for the Discriminant Function in the Naive Bayes Classifier

Classification can also be seen as implementing a set of **discriminant functions**, $g_i(x)$, $i=1, \dots, K$, such that we

$$\begin{aligned} & \text{Choose } C_i \text{ if } g_i(x) = \max_k g_k(x) \\ & \text{where } g_i(\mathbf{x}) = \ln(P(C_i|\mathbf{x})) = \ln(P(\mathbf{x}|C_i)P(C_i)). \end{aligned}$$

When there are **two classes** (C_1 and C_2), we have the **Bayesian decision rule**

$$\begin{aligned} & \text{Choose } C_1 \text{ if } P(C_1|x) > P(C_2|x) \parallel \text{Choose } \\ & C_1 \text{ if } P(x|C_1)P(C_1) > P(x|C_2)P(C_2) \parallel \text{Choose } C_1 \\ & \text{if } \ln(P(x|C_1)P(C_1)) > \ln(P(x|C_2)P(C_2)) \parallel \text{Choose } C_1 \text{ if } \\ & g_1(x) > g_2(x) \parallel \text{Choose } C_1 \text{ if } g_1(x) - g_2(x) > 0 \end{aligned}$$

When there are **two classes**, we can define a single discriminant

$$g(\mathbf{x}) = g_1(\mathbf{x}) - g_2(\mathbf{x})$$

and we

$$\text{Choose } \begin{cases} C_1 & \text{if } g(\mathbf{x}) > 0 \\ C_2 & \text{otherwise} \end{cases}$$

Exercise 1

Explicitly calculate the decision boundary for the two-class two-dimensional data.

Assume that the data likelihood for each class is a bivariate Gaussian distribution

$$P(\mathbf{x}|C_i) = \frac{1}{(2\pi)^{d/2} |\Sigma_i|^{1/2}} \exp\left\{-\frac{1}{2} (\mathbf{x} - \mu_i)^T \Sigma_i^{-1} (\mathbf{x} - \mu_i)\right\}$$

where

$\mu_1 = \begin{bmatrix} 3 \\ 6 \end{bmatrix}$, $\mu_2 = \begin{bmatrix} 3 \\ -2 \end{bmatrix}$, $\Sigma_1 = \begin{bmatrix} 1/2 & 0 \\ 0 & 2 \end{bmatrix}$, $\Sigma_2 = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$
 The inverse matrices are

$\Sigma_1^{-1} = \begin{bmatrix} 2 & 0 \\ 0 & 1/2 \end{bmatrix}$,
 $\Sigma_2^{-1} = \begin{bmatrix} 1/2 & 0 \\ 0 & 1/2 \end{bmatrix}$
 Assume equal prior probabilities $P(C_1) = P(C_2) = \frac{1}{2}$.

1. Compute the discriminant function (decision function).

Answer in board notes.

```
In [2]: import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
plt.style.use('bmh')
import scipy.stats as stats
```

```
In [3]: mu1 = [3, 6]
mu2 = [3, -2]

Sigma1 = np.array([[0.5, 0], [0, 2]])
Sigma2 = np.array([[2, 0], [0, 2]])

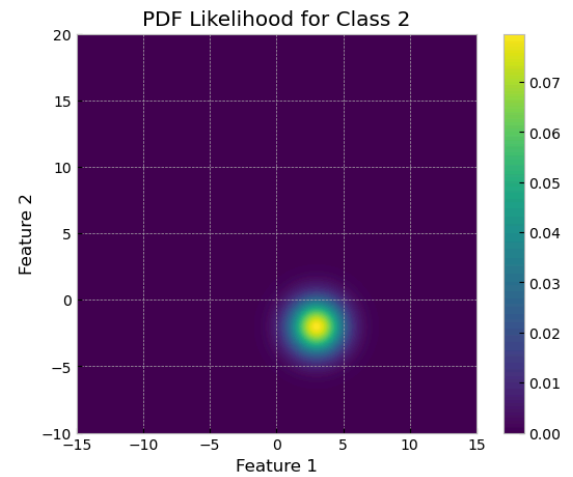
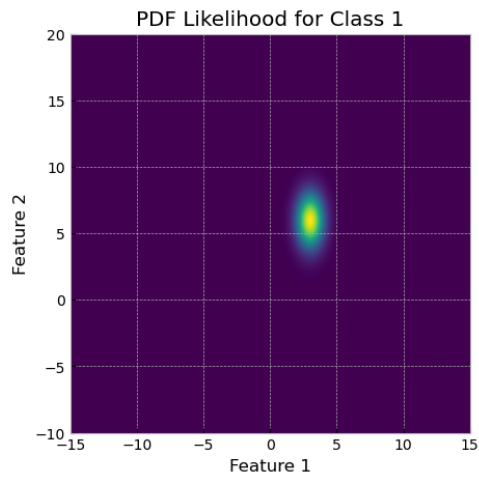
p1 = 0.5
p2 = 1-p1
```

```
In [4]: # Compute a grid of values for x and y
gridx = 15
gridy = 20
x = np.linspace(-gridx, gridx, 500)
y = np.linspace(-10, gridy, 500)
xm, ym = np.meshgrid(x, y)
X = np.flip(np.dstack([xm, ym]), axis=0) # grid of values

# Let's plot the probability density function (pdf) for each class
y1 = stats.multivariate_normal.pdf(X, mean=mu1, cov=Sigma1) #P(x|C1) - data
y2 = stats.multivariate_normal.pdf(X, mean=mu2, cov=Sigma2) #P(x|C2)

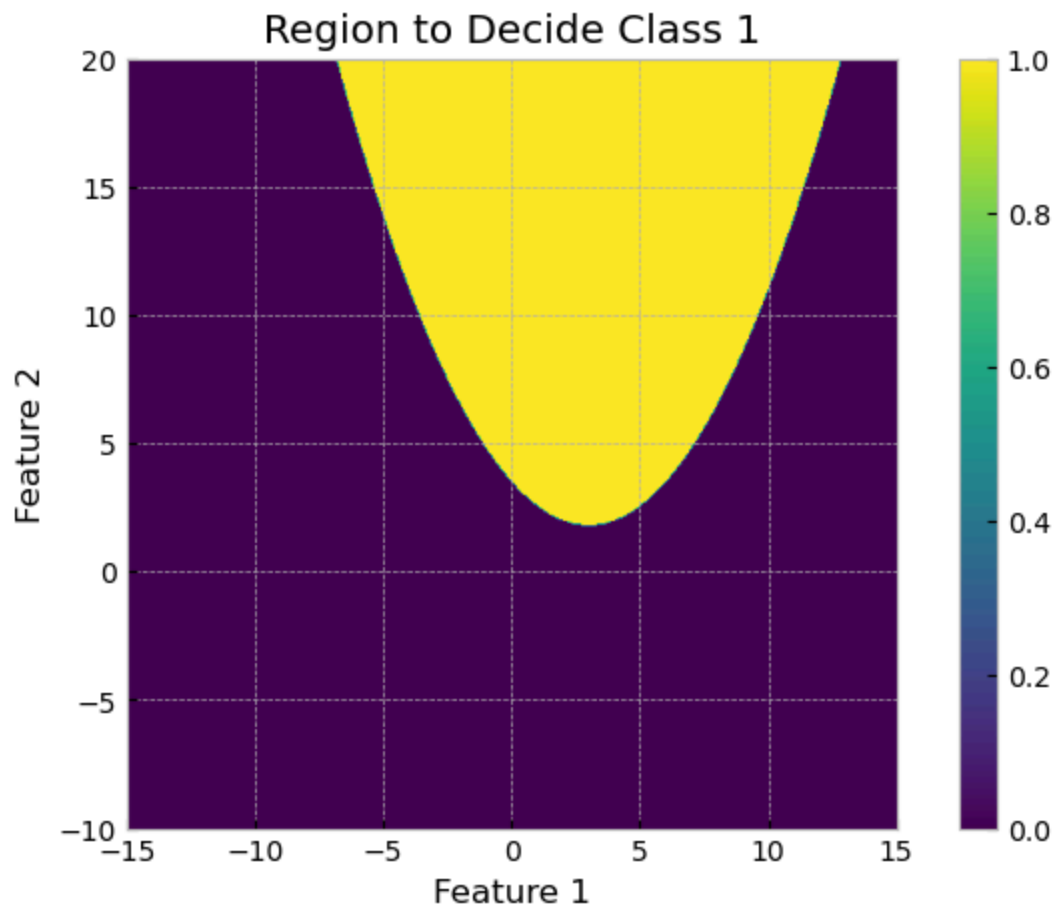
fig = plt.figure(figsize=(15, 5))
fig.add_subplot(1, 2, 1)
plt.imshow(y1, extent=[-gridx, gridx, -10, gridy])
plt.colorbar()
plt.xlabel('Feature 1'); plt.ylabel('Feature 2')
plt.title('PDF Likelihood for Class 1')

fig.add_subplot(1, 2, 2)
plt.imshow(y2, extent=[-gridx, gridx, -10, gridy])
plt.colorbar()
plt.xlabel('Feature 1'); plt.ylabel('Feature 2')
plt.title('PDF Likelihood for Class 2');
```



```
In [5]: # Let's take a look at the posterior distributions: they represent our class
pos1 = (y1*p1)/(y1*p1 + y2*p2) #  $P(C1|x)$  - posterior probability
pos2 = (y2*p2)/(y1*p1 + y2*p2) #  $P(C2|x)$ 

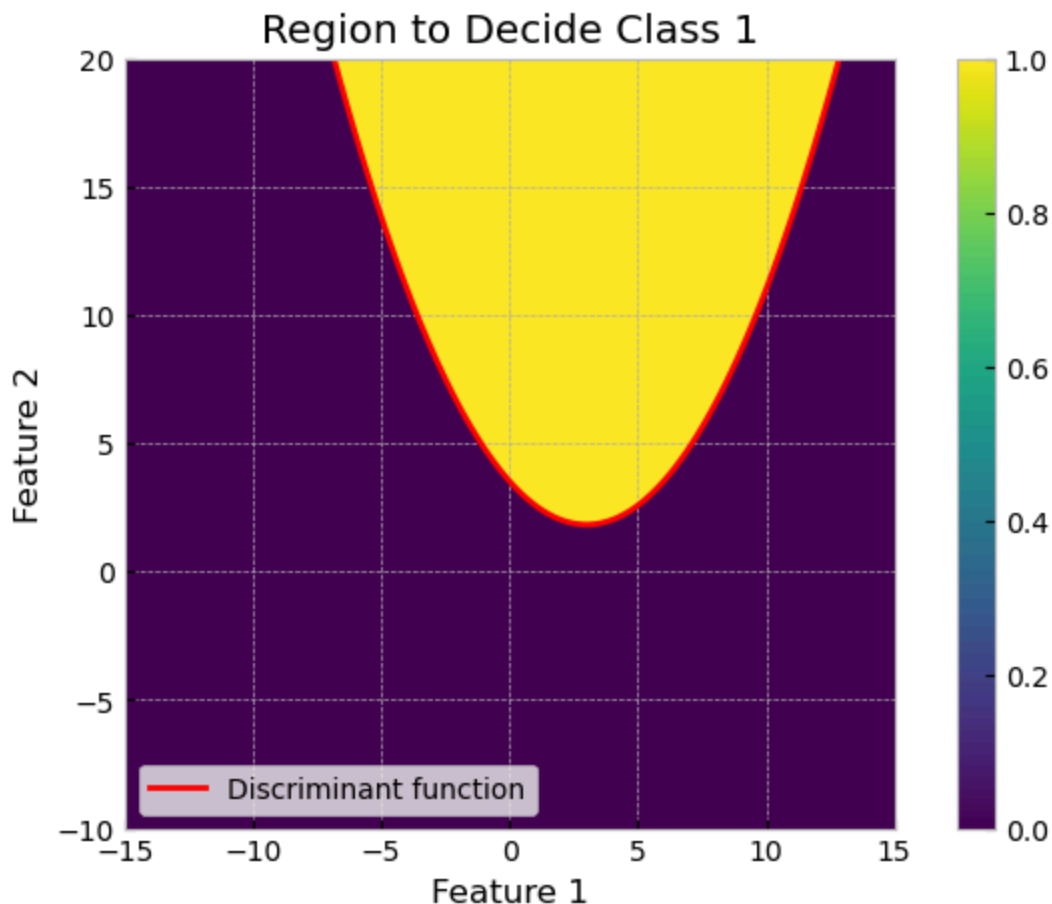
# Look at the decision boundary:
plt.figure(figsize=(8,5))
plt.imshow(pos1>pos2, extent=[-gridx,gridx,-10,gridy])
plt.colorbar()
plt.xlabel('Feature 1'); plt.ylabel('Feature 2')
plt.title('Region to Decide Class 1');
```



In [6]: `# Compare with the one derived on the blackboard – they are the same, as exp`

```
x1 = np.linspace(-6.8,12.8, 100)

plt.figure(figsize=(8,5))
plt.imshow(pos1>pos2, extent=[-gridx,gridx,-10,gridy])
plt.plot(x1, 3.514 - 1.125*x1 + 0.1875*x1**2, 'r', label='Discriminant funct')
plt.colorbar(); plt.legend()
plt.xlabel('Feature 1'); plt.ylabel('Feature 2')
plt.title('Region to Decide Class 1');
```



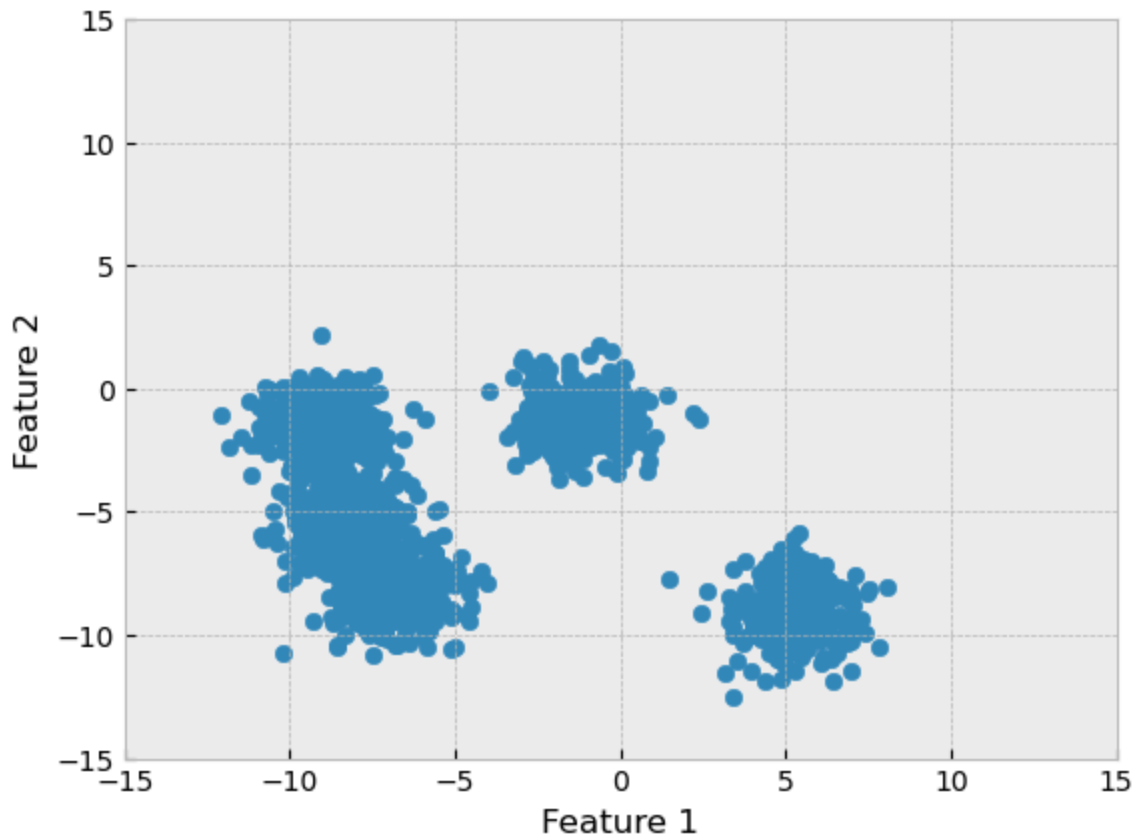
Mixture Models

What if the data for a *single class* looks like the plot below?

```
In [7]: from sklearn.datasets import make_blobs

data, _ = make_blobs(n_samples = 1500, centers = 5)

plt.scatter(data[:,0],data[:,1]); plt.axis([-15,15,-15,15])
plt.xlabel('Feature 1'); plt.ylabel('Feature 2');
```



If we assume a single Gaussian distribution, we would obtain a very poor estimate of the true underlying data likelihood.

We can better represent this data with a **mixture model**:

$$p(x|\Theta) = \sum_{k=1}^K \pi_k P(x|\Theta_k)$$

where $\Theta = \{\Theta_k\}_{k=1}^K$ are set of parameters that define the distributional form in the probabilistic model $P(\bullet|\Theta_k)$ and

$$\begin{aligned} 0 &\leq \pi_k \leq 1 \quad \& \quad \sum_k \pi_k = 1 \end{aligned}$$

Gaussian Mixture Models

A **Gaussian Mixture Model** or **GMM** is a probabilistic model that assumes a data likelihood to be a weighted sum of Gaussian distributions with unknown parameters.

$$p(\mathbf{x}|\Theta) = \sum_{k=1}^K \pi_k N(\mathbf{x}|\boldsymbol{\mu}_k, \Sigma_k)$$

where $\Theta = \{\pi_k, \boldsymbol{\mu}_k, \Sigma_k\}_{k=1}^K$, $0 \leq \pi_k \leq 1$ and $\sum_{k=1}^K \pi_k = 1$.

- When standard distributions (such as Gamma, Exponential, Gaussian, etc.) are not sufficient to characterize a *complicated* data likelihood, we can instead characterize it as the sum of weighted Gaussians distributions

- Another way that GMMs are most commonly used for is to partition data in subgroups

Modeling a Data Likelihood as a Gaussian Mixture Model

- GMMs can be used to learn a complex distribution that represent a dataset. Thus, it can be used within the probabilistic generative classifier framework to model complex data likelihoods.
- GMMs are also commonly used for **clustering**. Here a GMM is fit to a dataset with the goal of partitioning it into clusters.

Step 1

Describe the **observed data likelihood**, $\mathcal{L}^0$. As seen last class:

$$\mathcal{L}^0 = \prod_{i=1}^N \sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)$$

Step 2

Describe the log-likelihood function:

$$\mathcal{L} = \ln \left(\prod_{i=1}^N \sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k) \right) \iff \mathcal{L} = \sum_{i=1}^N \ln \left(\sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k) \right)$$

Step 3

Optimize for the parameters $\Theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$

$$\frac{\partial \mathcal{L}}{\partial \mu_k} = 0, \frac{\partial \mathcal{L}}{\partial \Sigma_k} = 0, \text{ and } \frac{\partial \mathcal{L}}{\partial \pi_k} = 0$$

but this is a difficult problem to maximize!

- A common approach for estimating the parameters of a GMM given a data set is by using the **Expectation-Maximization (EM) algorithm**.

Applications of Mixture Models

- **Data Representation and Inference.** Represent *any* data set using a complex distribution. You may want to describe your data with a distribution but the data does not fit into a standard form (e.g. Gamma, Gaussian, Exponential, etc.), then you can use a (Gaussian) Mixture Model to describe the data. Having a representation of your data in a distribution form is a very powerful tool that, other than having a model of the data, allows you infer about new samples, make predictions, etc.
 - Having a parametric form for a real world model, allows us to apply it in simulation and use it for designing/optimizing decision-making solutions.
- **Clustering.** Partition the data into groups. Note that in the GMMs formulation we did not add the concept of labels/targets. So GMMs are an **unsupervised learning** model. It represents the data with a very complex likelihood and then we can decompose that likelihood to partition the data into categories. This is also known as **clustering**.

Expectation-Maximization (EM) algorithm

Example: Censored Data

Consider the observation of i.i.d. samples $x_1, x_2, \dots, x_N$ from the data likelihood $p(\mathbf{x}|\Theta)$ and we want to estimate the parameters (using MLE approach, for example)

$$\arg_{\Theta} \max p(\mathbf{x}|\Theta) = \arg_{\Theta} \max \prod_{i=1}^N p(x_i|\Theta)$$

Now suppose the data samples $x_1, x_2, \dots, x_N$ are **censored**.

For example, suppose we observe i.i.d. samples, $x_1, x_2, \dots, x_N$, from some sensor $f(\mathbf{x})$. This sensor returns censored data in the form,

$$f(\mathbf{x}) = \begin{cases} x, & \text{if } x < a \\ a, & \text{if } x \geq a \end{cases}$$

This means that we see $x_1, x_2, \dots, x_m$ (less than a) and we do not see $x_{m+1}, x_{m+2}, \dots, x_N$ which are censored and set to a .

- An example of such censored data would be a scale that can only measure weight up to 120 lbs. Any measurements above 120 would be *censored* at 120.
- Given this censored data, we want to estimate the mean of the data as if the data was uncensored.

For this case, we can write our *observed* data likelihood as:

$$\begin{aligned} \mathcal{L}^0 &= \prod_{i=1}^m p(x_i|\Theta) \prod_{j=m+1}^N \\ \int_a^{\infty} p(x_j|\Theta) dx_j \end{aligned}$$

The data likelihood would be very difficult to maximize to solve for Θ .

If only we *knew* what the missing/censored data, the problem would be easy to solve!

Hidden Latent Variables

The **Expectation-Maximization** or **EM** algorithm is used to find the Maximum Likelihood Estimators (MLE) (or MAP estimators) for model parameters when data is incomplete, has missing data points, or has unobserved (hidden) latent variables (such as the case of censored data).

- For all of these cases, the MLE optimization is very difficult to obtain by simply taking the derivative and solve for the parameters.

Step 1 The first step of EM is to characterized the observed likelihood $\mathcal{L}^0$.

Step 2 Introduce *hidden latent variables* (also referred to as *hidden variables*) that simplify the observed data likelihood, $\mathcal{L}$.

Step 3 Use the hidden variables to define the *complete likelihood* $\mathcal{L}^c$.

With this, we build the EM optimization function:

$$\arg_{z, \Theta} \max Q(\Theta, \Theta^t)$$

where

$$Q(\Theta, \Theta^t) = E[\ln(\mathcal{L}^c)|X, \Theta^t]$$

$E[\bullet]$ denotes expected value and t denotes *iteration*. At $t=0$, we start with random values for the parameters Θ .

Once we have this, the EM algorithm will iterate between the E-step and the M-step:

- E-step (Expectation step)** Estimate the hidden variables. While holding Θ fixed, find the variables z that maximize $E[\ln(\mathcal{L}^c)]$.
- M-step (Maximization step)** Estimate the parameters of the complete data likelihood $\mathcal{L}^c$. While holding the newly found variables z , find the best values for the parameters Θ that maximize $E[\ln(\mathcal{L}^c)]$.

and it keeps iterating between E-step and M-step until convergence or until a certain number of iterations is reached.

Alternating Optimization

- EM is an **alternating optimization** algorithm, as it alternates between E-step and M-step in order to find the hidden variables and best set of parameters, respectively.
 - In the first step ($t=0$), EM will start with a random guess for the value of the parameters Θ in order to perform the E-step.
 - What is the problem with alternating optimization algorithms in general?
 - EM is a general algorithm that can be applied to a variety of problems (not just the examples we are learning today).
 - EM is heavily tied with Maximum Likelihood Estimation (MLE). It is commonly used to simplify difficult MLE problems.
 - It was originally introduced by Dempster, Laird, and Rubin in 1977 in a paper called "[Maximum Likelihood from Incomplete Data via the EM Algorithm](#)".
-

Optimizing GMM with the EM Algorithm

The observed data likelihood for a Gaussian Mixture Model (GMM) is

$$\mathcal{L}^0 = \prod_{i=1}^N \sum_{k=1}^K \pi_k \mathcal{N}(x_i | \mu_k, \Sigma_k)$$

What hidden variables can we add to simplify this problem?

- In this example, a hidden variable can be the label of the Gaussian from which x_i was drawn from.

$$z_i: \text{label of the Gaussian from which } x_i \text{ was drawn from}$$

If we know z_i , then for each data point we know which Gaussian it was drawn from along with its respective parameter μ_{z_i} and Σ_{z_i} and its respective weight π_{z_i} . So, each data point would have been drawn from $\pi_{z_i} \mathcal{N}(x_i | \mu_{z_i}, \Sigma_{z_i})$.

Then, assuming we have $\{z_i\}_{i=1}^N$, we can write the complete data likelihood:

$$\mathcal{L}^c = \prod_{i=1}^N \pi_{z_i} \mathcal{N}(x_i | \mu_{z_i}, \Sigma_{z_i})$$

Now we can iterate between the **E-step** and **M-step** of the EM algorithm until we find convergence or we have reached a threshold for a number of iterations.

Optimization Function

We can now extend the optimization function:

$$\begin{aligned} Q(\Theta, \Theta^t) &= E[\ln(\mathcal{L}^c) | X, \Theta^t] \quad \&= \\ \sum_{\mathbf{z}} \ln(\mathcal{L}^c) P(\mathbf{z} | X, \Theta^t) \quad \&= \sum_{i=1}^K \ln(\mathcal{L}^c) P(\mathbf{z}_i | \mathbf{x}_i, \Theta^t) \end{aligned}$$

E-step (Expectation Step)

In order to complete the E-STEP, we need to know how to compute

$$P(\mathbf{z}_i | \mathbf{x}_i, \Theta^t).$$

- This is the posterior probability of the label z_i for data sample x_i .
- So we want to assign the label z_i to the data sample x_i for which the posterior probability is maximized (just like in Naive Bayes classification).

Recall from **Bayes' Rule**: for two non-empty events A and B , $P(A|B) =$

$$\frac{P(B|A)P(A)}{P(B)}.$$
 We can use this theorem to rewrite

$$P(\mathbf{z}_i | \mathbf{x}_i, \Theta^t):$$

$$\begin{aligned} P(\mathbf{z}_i | \mathbf{x}_i, \Theta^t) &= \\ \frac{P(\mathbf{x}_i | \mathbf{z}_i, \Theta^t) P(\mathbf{z}_i | \Theta^t)}{P(\mathbf{x}_i | \Theta^t)} \quad \&= \frac{P(\mathbf{x}_i | \mathbf{z}_i, \Theta^t)}{\sum_{i=1}^K P(\mathbf{x}_i | \mathbf{z}_i, \Theta^t)} \\ &= C_{ik} \end{aligned}$$

This is called the **memberships** or **responsibilities** matrix, which contains the label assignment for point x_i in each Gaussian component k .

- In the E-STEP we estimate the membership matrix $C_{ik} = P(\mathbf{z}_i | \mathbf{x}_i, \Theta^t)$. This matrix is of size $N \times K$ that contains the likelihoods of each point belonging in each one of the Gaussians.
- A good check when implementing this matrix is to make sure that the sum of the rows are equal to 1!

This completes the Expectation step (E-step) in EM. Now, we derive the update equations for the parameters $\Theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$ in the Maximization step.

M-step (Maximization Step)

In the **M-step**, we are going to use (and hold constant) the membership matrix C_{ik} we learned from the E-step.

We will now estimate the new set of parameters $\Theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^K$ that maximize $Q(\Theta, \Theta^t)$, i.e.

$$\arg_{\Theta} \max Q(\Theta, \Theta^t)$$

Without loss of generality, let's assume that the covariance matrices are isotropic:

$\Sigma_k = \sigma_k^2 \mathbf{I}$, then we can rewrite it as:

$$\begin{aligned} Q(\Theta, \Theta^t) &= \sum_{i=1}^K \ln(\mathcal{L}^c) \\ &P(\mathbf{z}_i | \mathbf{x}_i, \Theta^t) \parallel \sum_{i=1}^K \ln \left(\prod_{k=1}^N \pi_{z_i} G(\mathbf{x}_i | \mu_{z_i}, \Sigma_{z_i}) \right) \\ &P(\mathbf{z}_i | \mathbf{x}_i, \Theta^t) \parallel \sum_{k=1}^K \ln \left(\prod_{i=1}^N \pi_k G(\mathbf{x}_i | \mu_k, \Sigma_k) \right) P(\mathbf{z}_i = k | \mathbf{x}_i, \Theta^t) \parallel \\ &= \sum_{k=1}^K \sum_{i=1}^N \left(\ln(\pi_k) + \right. \\ &\left. \ln \left(G(\mathbf{x}_i | \mu_k, \Sigma_k) \right) \right) C_{ik} \parallel \sum_{k=1}^K \\ &\sum_{i=1}^N \left(\ln(\pi_k) - \frac{d}{2} \ln(2\pi) - \frac{d}{2} \ln(\sigma_k^2) - \frac{1}{2\sigma_k^2} \right. \\ &\left. \left\| \mathbf{x}_i - \mu_k \right\|_2^2 \right) C_{ik} \end{aligned}$$

So, now we want to solve:

$$\begin{aligned} \frac{\partial Q(\Theta, \Theta^t)}{\partial \mu_k} &= 0 \parallel \frac{\partial Q(\Theta, \Theta^t)}{\partial \sigma_k^2} = 0 \parallel \frac{\partial Q(\Theta, \Theta^t)}{\partial \pi_k} = 0 \end{aligned}$$

Solving for μ_k

$$\begin{aligned} 0 &= \frac{\partial Q(\Theta, \Theta^t)}{\partial \mu_k} \parallel 0 = \sum_{i=1}^N \left(\frac{1}{\sigma_k^2} (\mathbf{x}_i - \mu_k) \right) C_{ik} \parallel 0 = \sum_{i=1}^N (\mathbf{x}_i - \mu_k) C_{ik} \parallel 0 = \sum_{i=1}^N \mathbf{x}_i C_{ik} - \sum_{i=1}^N \mu_k C_{ik} \parallel \\ &\sum_{i=1}^N \mathbf{x}_i C_{ik} = \mu_k \sum_{i=1}^N C_{ik} \parallel \mu_k = \frac{\sum_{i=1}^N \mathbf{x}_i C_{ik}}{\sum_{i=1}^N C_{ik}} \end{aligned}$$

- We can interpret this as a **weighted mean**, where the weights represent the likelihood of each point belonging to cluster k .

Solving for σ_k^2

$$\begin{aligned} 0 &= \frac{\partial Q(\Theta, \Theta^t)}{\partial \sigma_k^2} \parallel 0 = \sum_{i=1}^N \left(-\frac{d}{2\sigma_k^2} + \frac{1}{2\sigma_k^4} \left\| \mathbf{x}_i - \mu_k \right\|_2^2 \right) C_{ik} \parallel \frac{d}{2\sigma_k^2} \sum_{i=1}^N C_{ik} = \frac{1}{2\sigma_k^4} \sum_{i=1}^N C_{ik} \left\| \mathbf{x}_i - \mu_k \right\|_2^2 \parallel \\ &d \sigma_k \sum_{i=1}^N C_{ik} = \sum_{i=1}^N C_{ik} \left\| \mathbf{x}_i - \mu_k \right\|_2^2 \parallel \sigma_k^2 = \frac{\sum_{i=1}^N C_{ik} \left\| \mathbf{x}_i - \mu_k \right\|_2^2}{\sum_{i=1}^N C_{ik}} \end{aligned}$$

- Again, assuming that the covariance of each cluster is isotropic, that is, $\Sigma_k = \sigma_k^2 \mathbf{I}$ (where Σ_k is a $d \times d$ matrix.)
- For the variance, this can be interpreted as how much each data point contributes to the k -th cluster.

Solving for π_k

For π_k we have the constraint that $\sum_{k=1}^K \pi_k = 1$. Then we need to incorporate it into $Q(\Theta, \Theta^t)$ using **Lagrange Multipliers**.

$$\begin{aligned} Q_{\pi}(\Theta, \Theta^t) &= Q(\Theta, \Theta^t) + \lambda \left(1 - \sum_{k=1}^K \pi_k\right) \end{aligned}$$

where λ is called the Lagrange multiplier.

$$\begin{aligned} 0 &= \frac{\partial Q_{\pi}(\Theta, \Theta^t)}{\partial \pi_k} \quad 0 = \sum_{i=1}^N C_{ik} \frac{1}{\pi_k} - \lambda \quad 0 = \frac{\sum_{i=1}^N C_{ik}}{\lambda \pi_k} \end{aligned}$$

Since $\sum_k \pi_k = 1$, we have that:

$$\begin{aligned} \sum_{k=1}^K \pi_k &= 1 \quad \sum_{k=1}^K \frac{\sum_{i=1}^N C_{ik}}{\lambda \pi_k} = 1 \quad \lambda = \sum_{i=1}^N \sum_{k=1}^K C_{ik} \end{aligned}$$

Plugging it back, we find:

$$\begin{aligned} \pi_k &= \frac{\sum_{i=1}^N C_{ik}}{\lambda} \quad \pi_k = \frac{\sum_{i=1}^N C_{ik}}{\sum_{i=1}^N \sum_{k=1}^K C_{ik}} \quad \pi_k = \frac{\sum_{i=1}^N C_{ik}}{\sum_{i=1}^N 1} = \frac{\sum_{i=1}^N C_{ik}}{N} \end{aligned}$$

- The π_k 's are summing up all its responsibilities over all data points.

Pseudo-Code

We now have everything we need to implement the EM algorithm for Gaussian Mixtures.

- The pseudo-code for the algorithm is:

```
In [1]: from IPython.display import Image
Image('figures/PseudoCode_EMforGMM.png',width=700)
```

Out[1]: **Algorithm 1** EM for Gaussian Mixture Model

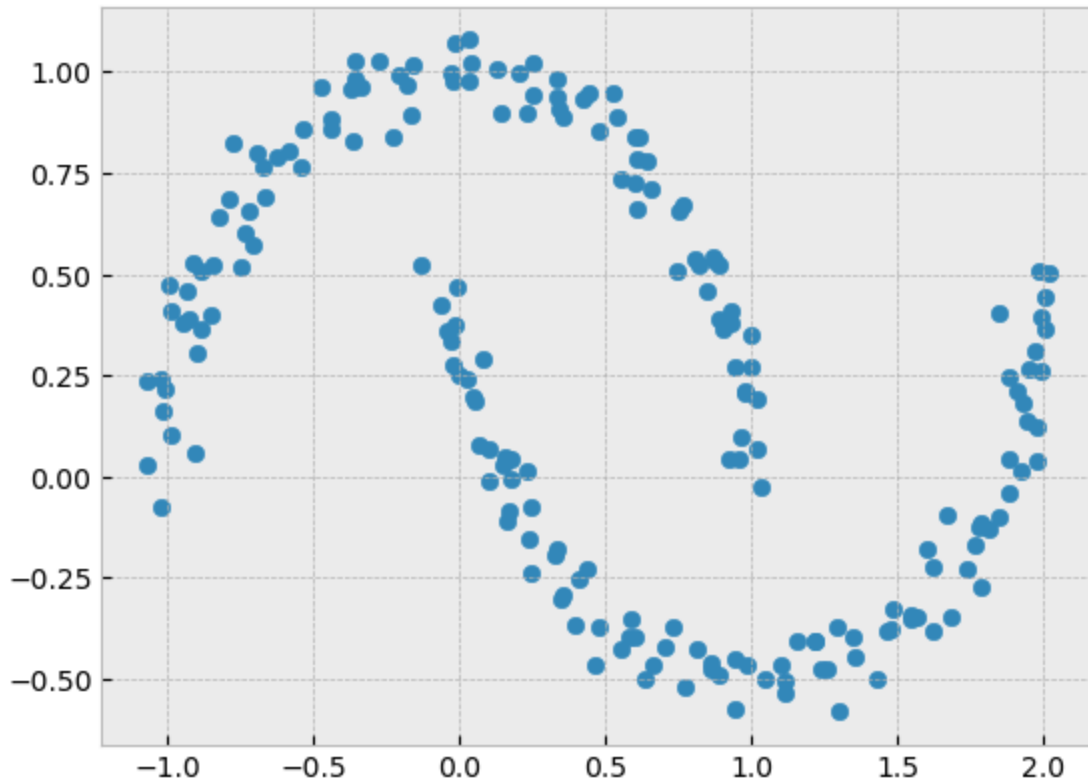
- 1: **INPUT:** Training data $\mathbf{X}$, number of Gaussian terms K
 - 2: Initialize all parameters (μ_k , Σ_k and π_k)
 - 3: $t=1$
 - 4: **while** convergence not yet reached **OR** maximum number of iterations reached **do**
 - 5: **E-STEP:**
 Compute $C_{ik} = \frac{\pi_{z_i}^t P(\mathbf{x}_i | \mu_{z_i}^t, \Sigma_{z_i}^t)}{\sum_{z_i=1}^K \pi_{z_i}^t P(\mathbf{x}_i | \mu_{z_i}^t, \Sigma_{z_i}^t)}$ for every x_i and k .
 C is a $N \times k$ matrix, where each row sums to 1
 - 6: **M-STEP:**
 - (1) Update μ_k for all k . $\mu_k^{t+1} = \frac{\sum_{i=1}^N C_{ik} x_i}{\sum_{i=1}^N C_{ik}}$, where μ_k is a $d \times 1$, and U is a $d \times k$ matrix.
 - (2) Update σ_k^2 for all k . $\sigma_k^{2^{t+1}} = \frac{\sum_{i=1}^N C_{ik} \|x_i - \mu_k^t\|_2^2}{d \sum_{i=1}^N C_{ik}}$, where σ_k^2 is a $d \times d$, and Σ is a $d \times d \times k$ tuple.
 - (3) Update π_k for all k . $\pi_k^{t+1} = \frac{\sum_{i=1}^N C_{ik}}{N}$, where π_k is a scalar, and Π is a $d \times 1$ vector.
 - 7: $t = t + 1$
 - 8: Check convergence criteria
 - 9: **end while**
 - 10: **OUTPUT:** C_{ik} , μ_k , Σ_k and π_k
-

Example: GMM as Density Estimation

Consider some data generated from Scikit-Learn's `make_moons` function:

```
In [8]: from sklearn.datasets import make_moons
from matplotlib.patches import Ellipse

Xmoon, ymoon = make_moons(200, noise=.05, random_state=0)
plt.scatter(Xmoon[:, 0], Xmoon[:, 1]);
```



Let's create a helper function that will help us visualize the locations and shapes of the GMM clusters by drawing ellipses based on the GMM output.

- Let's not worry about the implementation of this helper function.

```
In [9]: # Code from "Python Data Science Handbook"
# https://jakevdp.github.io/PythonDataScienceHandbook/05.12-gaussian-mixture

def draw_ellipse(position, covariance, ax=None, **kwargs):
    """Draw an ellipse with a given position and covariance"""
    ax = ax or plt.gca()

    # Convert covariance to principal axes
    if covariance.shape == (2, 2):
        U, s, Vt = np.linalg.svd(covariance)
        angle = np.degrees(np.arctan2(U[1, 0], U[0, 0]))
        width, height = 2 * np.sqrt(s)
    else:
        angle = 0
        width, height = 2 * np.sqrt(covariance)

    # Draw the Ellipse
    for nsig in range(1, 4):
        ax.add_patch(Ellipse(position, nsig * width, nsig * height,
                              angle=angle, **kwargs))

def plot_gmm(gmm, X, label=True, ax=None):
    ax = ax or plt.gca()
    labels = gmm.fit(X).predict(X)
    if label:

```

```

        ax.scatter(X[:, 0], X[:, 1], c=labels, s=40, cmap='viridis', zorder=
else:
        ax.scatter(X[:, 0], X[:, 1], s=40, zorder=2)
ax.axis('equal')

w_factor = 0.2 / gmm.weights_.max()
for pos, covar, w in zip(gmm.means_, gmm.covariances_, gmm.weights_):
    draw_ellipse(pos, covar, alpha=w * w_factor)

```

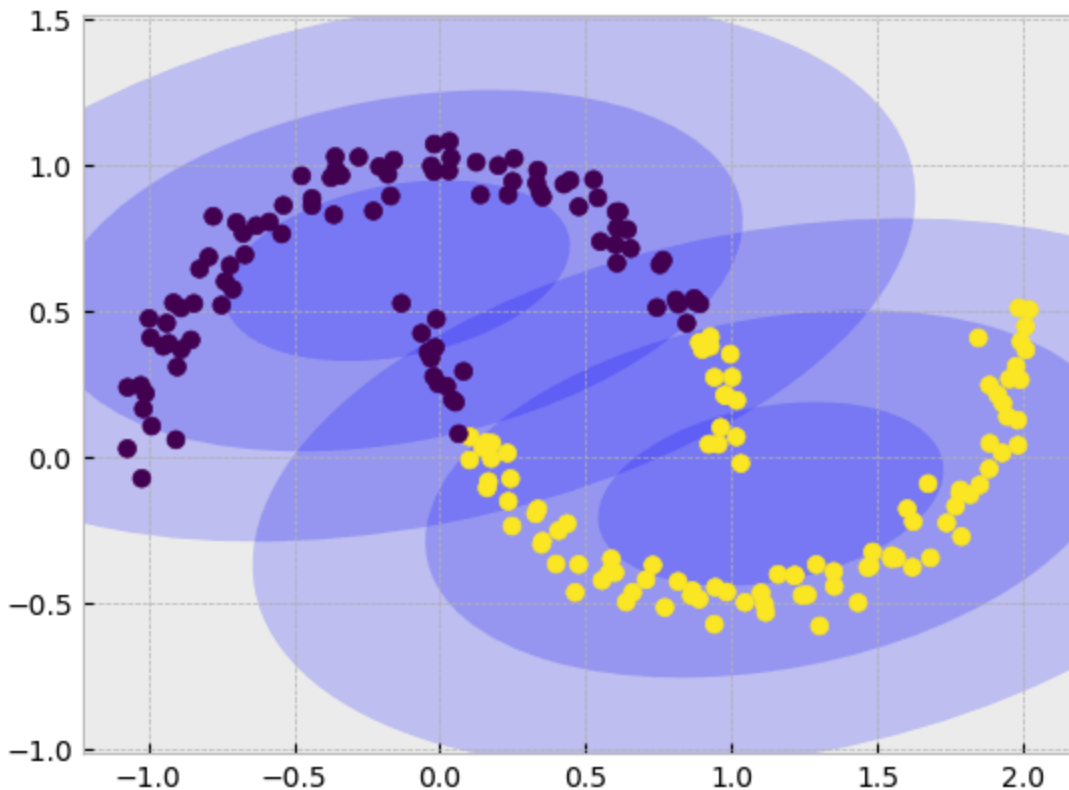
If we try to fit this with a two-component GMM viewed as a clustering model, the results are not particularly useful:

```

In [10]: from sklearn.mixture import GaussianMixture

GMM = GaussianMixture(n_components=2, covariance_type='full', random_state=0)
plot_gmm(GMM, Xmoon)

```

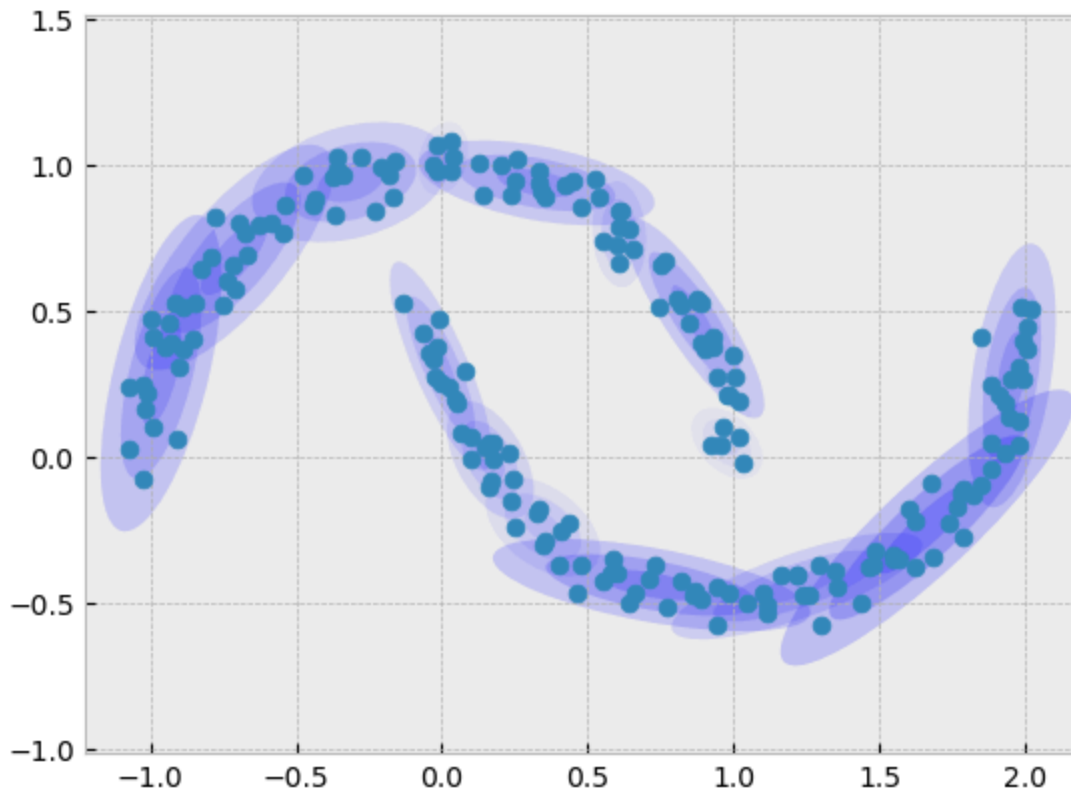


But if we instead use many more components and ignore the cluster labels, we find a fit that is much closer to the input data:

```

In [11]: GMM16 = GaussianMixture(n_components=16, random_state=0).fit(Xmoon)
plot_gmm(GMM16, Xmoon, label=False)

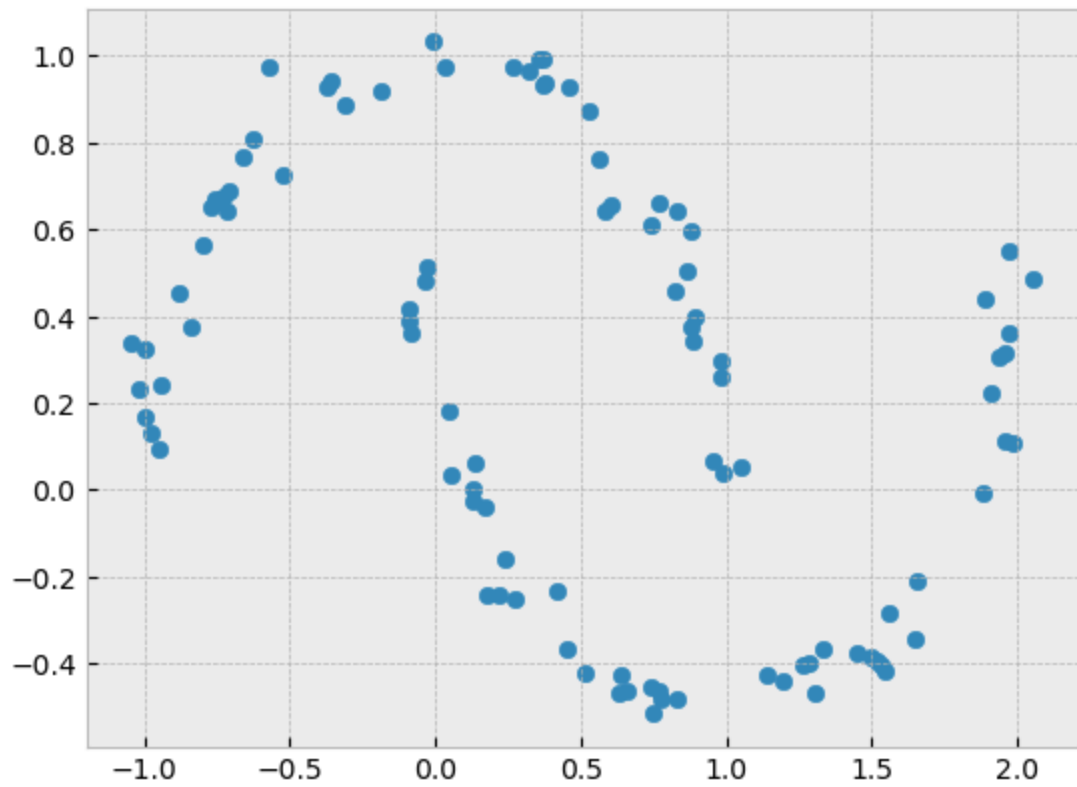
```



Here the mixture of 16 Gaussians serves not to find separated clusters of data, but rather to model the overall distribution of the input data. This is a generative model of the distribution, meaning that the GMM gives us the recipe to generate new random data distributed similarly to our input.

For example, here are 400 new points drawn from this 16-component GMM fit to our original data:

```
In [12]: Xnew = GMM16.sample(100)
plt.scatter(Xnew[0][:, 0], Xnew[0][:, 1]);
```

GMM is convenient as a flexible means of modeling an arbitrary multi-dimensional distribution of data.
